

Dokumentation StarFocus

SYT PROJEKT SOMMERSEMESTER
IDA KALEA RITTHAMMER

Inhaltsverzeichnis

Literaturverzeichnis	2
Einführung	3
Was ist StarFocus?	3
Bauteile	4
Microcontroller	4
Sensor	4
Lipo Akku	6
Base für den Microcontroller	6
Zusammenbau	7
Code	8
Testcode	8
Finaler code	9
Würfel	12
OnShape design	12
3D gedruckt	13
Node.js	14
Installation	14
Verbinden	14

Literaturverzeichnis

Elektronik Kompendium. (kein Datum). Von https://www.elektronik-kompendium.de/sites/praxis/bauteil_mpu6050-imu.htm abgerufen

Pkcell. (kein Datum). Von <https://www.pkcell.com/de/produkt/lp552035-350mah-3-7v/> abgerufen

SeedStudio Wiki. (kein Datum). Von https://wiki.seeedstudio.com/xiao_esp32c6_getting_started/ abgerufen

Einführung

Was ist StarFocus?

Viele Menschen haben beim Lernen oder Arbeiten Schwierigkeiten, ihre Zeit effektiv zu nutzen und fokussiert zu bleiben. Eine bekannte Methode zur Verbesserung der Konzentration ist die Pomodoro Technik, bei der Arbeitsphasen und Pausen strukturiert eingeteilt werden.

Das Projekt StarFocus soll diese Methode unterstützen. Dabei wird eine Kombination aus einer App und einem physischen Würfel entwickelt. Durch das Drehen des Würfels kann der Benutzer einfach auswählen, wie lange er sich konzentrieren möchte. In der App kann man anschließend einen entsprechenden Fokus-Timer einstellen und schon mit fokussieren loslegen.

In dieser Dokumentation wird spezifisch beschrieben, wie der haptische Teil von StarFocus erstellt wurde. „Der Würfel“ ist ein 3D gedrucktes Gehäuse mit einem MPU6050 Sensor eingebaut, der es ermöglicht das Drehen des Würfels zu erfassen und per WLAN an den Node.js Server zu schicken. Dadurch verändert sich die Auswahl des Bereiches, auf den sich der Benutzer fokussieren will.

Bauteile

Microcontroller

Als Microcontroller haben wir einen Seed Studio XIAO ESP32C6 benutzt.



Der Microcontroller unterstützt 2,4 GHz WiFi 6 und ist verhältnismäßig zu anderen Optionen um einiges kleiner, was gut für unseren Würfel ist, da er dadurch nicht zu groß geworden ist.

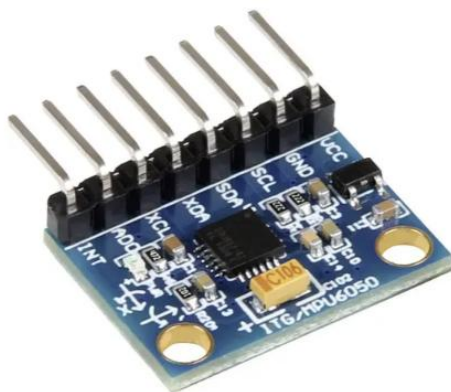
Er unterstützt auch ZigBee, was wir anfangs überlegt haben zu nutzen uns allerdings dann dagegen entschieden haben.

Er braucht zu übertragen des Codes ein USB-C zu USB-Kabel.

(SeedStudio Wiki, kein Datum)

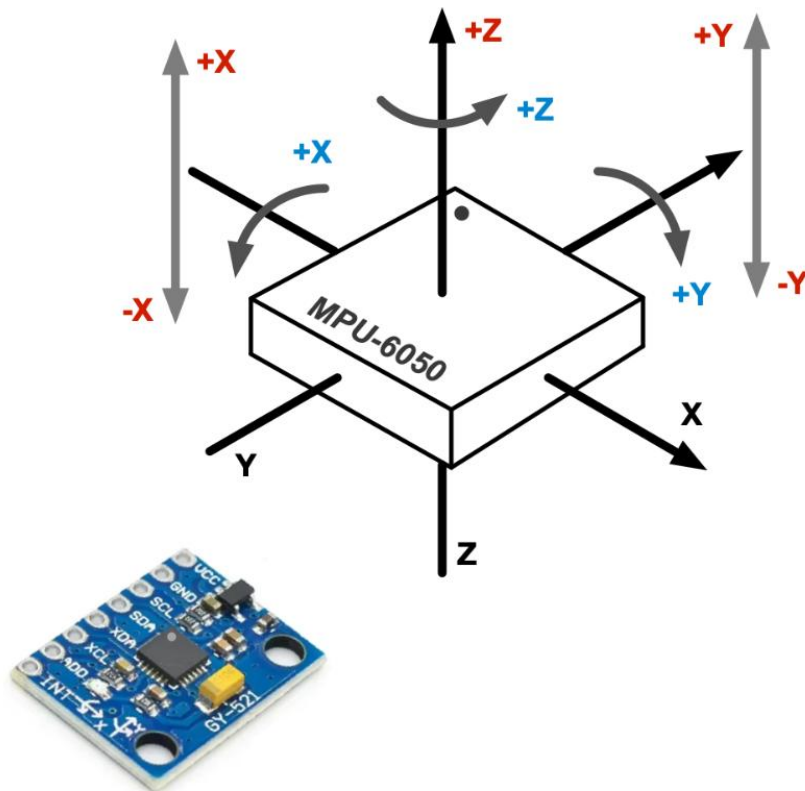
Sensor

Als Sensor haben wir einen MPU6050 benutzt.



Der MPU6050 Sensor kombiniert einen 3-Achsen-Beschleunigungssensor und ein 3-Achsen-Gyroskop, um präzise Bewegungs- und Drehungsdaten zu erfassen.

Dies haben wir in dem Würfel benutzt, um erfolgreich zu erfassen, wenn der Würfel auf eine neue Seite gedreht wurde.



Das Gyroskop im MPU-6050 misst die Winkelgeschwindigkeit in Grad pro Sekunde ($^{\circ}/s$) entlang der drei Achsen (X, Y, Z). Es misst die relative Drehbewegung, also wie schnell sich das Objekt um eine Achse dreht.

- In einem typischen dreidimensionalen Koordinatensystem entspricht die **X-Achse** der horizontalen Achse, die von links nach rechts verläuft, wenn das Objekt flach liegt und sich nicht bewegt.
- In einem typischen dreidimensionalen Koordinatensystem entspricht die **Y-Achse** der vertikalen Achse, die von hinten nach vorne verläuft, wenn das Objekt flach liegt und sich nicht bewegt.
- In einem typischen dreidimensionalen Koordinatensystem entspricht die **Z-Achse** der Achse, die von unten nach oben verläuft, wenn das Objekt flach liegt und sich nicht bewegt. Das ist die Achse, die senkrecht zur Oberfläche des Sensors steht.

(Elektronik Kompendium, kein Datum)

Lipo Akku

Wir haben als Akku eine Lithiumbatterie Pkcell LP503562 gewählt.



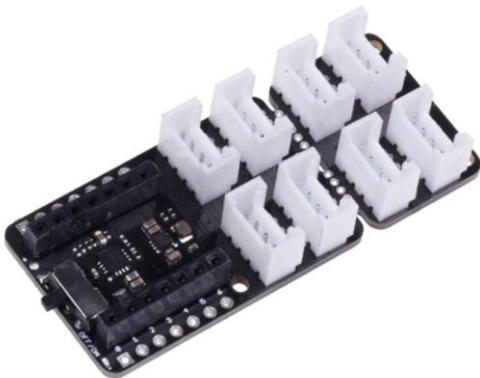
Die Lithiumbatterie Pkcell LP503562 liefert langanhaltende und zuverlässige Energie für verschiedene Geräte. Ihre kompakte Größe fügt sich nahtlos in Geräte ein und bietet eine außergewöhnliche Energiedichte, ohne die Tragbarkeit zu beeinträchtigen.

Dadurch, dass die Lithiumbatterie so klein war, hat sie perfekt in den Würfel gepasst. Mit der Lithiumbatterie bekommt der Microcontroller auch Energie, wenn er nicht an einem externen Gerät angeschlossen ist.

(Pkcell, kein Datum)

Base für den Microcontroller

Als base haben wir eine SEEED STUDIO GROVE BASE FOR XIAO benutzt.



Das SEEED STUDIO GROVE BASE FOR XIAO ist eine Plug-and-Play-Erweiterungsplatine für die Seeed Studio XIAO-Serie. Dank des integrierten Batteriemanagement-Chips und der Batterieanschlussfläche können Sie Ihr Seeed Studio XIAO ganz einfach mit einer Lithium-Batterie betreiben und wieder aufladen.

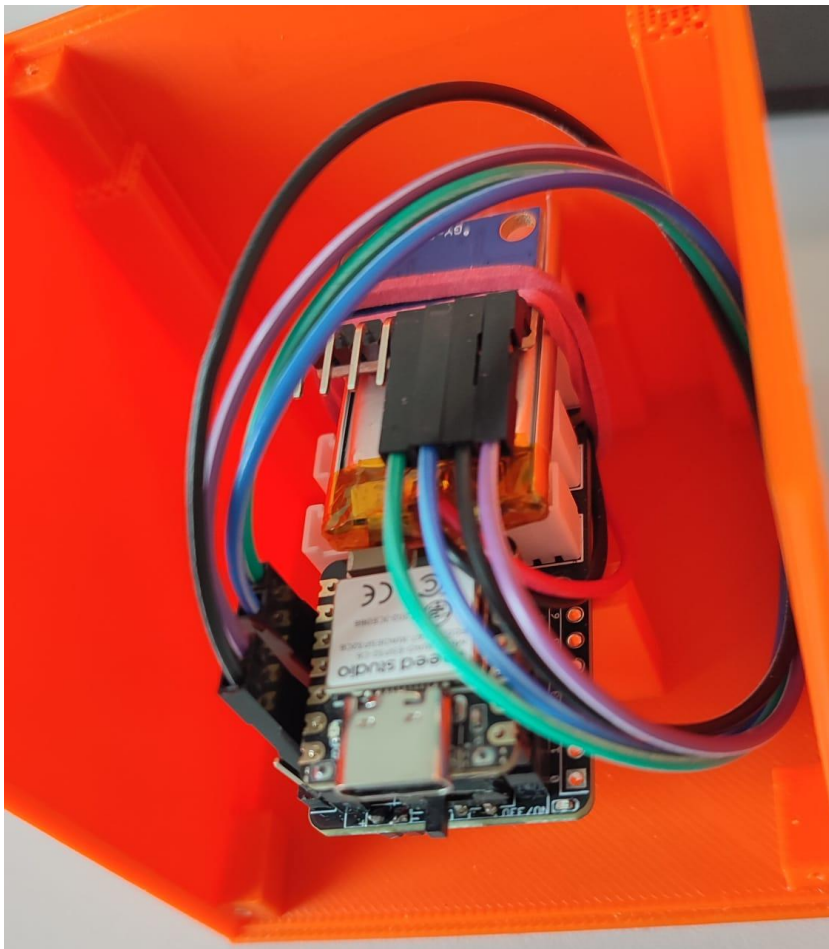
Mit diesem Bauteil konnten wir ohne Probleme die Lithium Batterie befestigen.

Zusammenbau

Verkabelung zwischen dem Microcontroller und dem Sensor.

MPU6050	ESP32-C6
VCC	3.3V
GND	GND
SCL	GPIO20
SDA	GPIO19

Endergebnis:



Code

Testcode

```
#include <Wire.h>
#include <MPU6050.h>

MPU6050 mpu;

String lastSide = "";

void setup() {
  Serial.begin(115200);
  delay(1000);

  Serial.println("Starte MPU6050 Test...");

  Wire.begin(19, 20);
  mpu.initialize();

  if (mpu.testConnection()) {
    Serial.println("MPU6050 Verbindung erfolgreich!");
  } else {
    Serial.println("FEHLER: MPU6050 nicht gefunden!");
    while (1);
  }
}

String getSide(int ax, int ay, int az) {
  int threshold = 15000;

  if (az > threshold) return "Seite 1 (Learning)";
  if (az < -threshold) return "Seite 6 (Default)";
  if (ay > threshold) return "Seite 2 (Gardening)";
  if (ay < -threshold) return "Seite 3 (Working)";
  if (ax > threshold) return "Seite 4 (Cooking)";
  if (ax < -threshold) return "Seite 5 (House work)";

  return "Unbekannt";
}
```

```

void loop() {
    int16_t ax, ay, az;

    mpu.getAcceleration(&ax, &ay, &az);

    String currentSide = getSide(ax, ay, az);

    if (currentSide != lastSide && currentSide != "Unbekannt") {
        Serial.println("Neue Seite erkannt:");
        Serial.println(currentSide);
        Serial.println("-----");

        lastSide = currentSide;
    }

    delay(300);
}

```

Finaler code

```

#include <Wire.h>
#include <MPU6050.h>
#include <WiFi.h>
#include <HTTPClient.h>

MPU6050 mpu;

const char* ssid = "ssid eingeben";
const char* password = "passwort eingeben";
const char* serverName = "http://Laptop Ip:port/update";
String lastSide = "";

void setup() {
    Serial.begin(115200);
    delay(1000);
    Serial.println("ESP startet...");
    // I2C starten
    Wire.begin(19, 20);

```

```

mpu.initialize();
if (!mpu.testConnection()) {
    Serial.println("MPU6050 Verbindung fehlgeschlagen!");
    while (1);
}

Serial.println("MPU6050 bereit!");
WiFi.begin(ssid, password);
Serial.print("Verbinde mit WLAN");

int tries = 0;

while (WiFi.status() != WL_CONNECTED && tries < 30) {
    delay(1000);
    Serial.print(".");
    tries++;
}

Serial.println();

if (WiFi.status() == WL_CONNECTED) {
    Serial.println("WLAN verbunden!");
    Serial.print("IP: ");
    Serial.println(WiFi.localIP());
} else {
    Serial.print("Fehlercode: ");
    Serial.println(WiFi.status());
}
}

String getSide(int ax, int ay, int az) {
    int threshold = 15000;

    if (az > threshold) return "study";
    if (az < -threshold) return "code";
    if (ay > threshold) return "reading";
    if (ay < -threshold) return "work";
    if (ax > threshold) return "creative";
    if (ax < -threshold) return "exercise";
}

```

```

    return "Unknown";
}

void sendData(String side) {
    if (WiFi.status() == WL_CONNECTED) {

        HTTPClient http;
        http.begin(serverName);
        http.addHeader("Content-Type", "application/json");

        String json = "{\"side\": \"" + side + "\"}";

        int httpResponseCode = http.POST(json);

        Serial.print("Gesendet: ");
        Serial.print(side);
        Serial.print(" | Code: ");
        Serial.println(httpResponseCode);

        http.end();
    }
}

void loop() {
    int16_t ax, ay, az;
    mpu.getAcceleration(&ax, &ay, &az);
    String currentSide = getSide(ax, ay, az);

    if (currentSide != lastSide && currentSide != "Unknown") {
        Serial.print("Neue Seite erkannt: ");
        Serial.println(currentSide);

        sendData(currentSide);
        lastSide = currentSide;
    }

    delay(500);
}

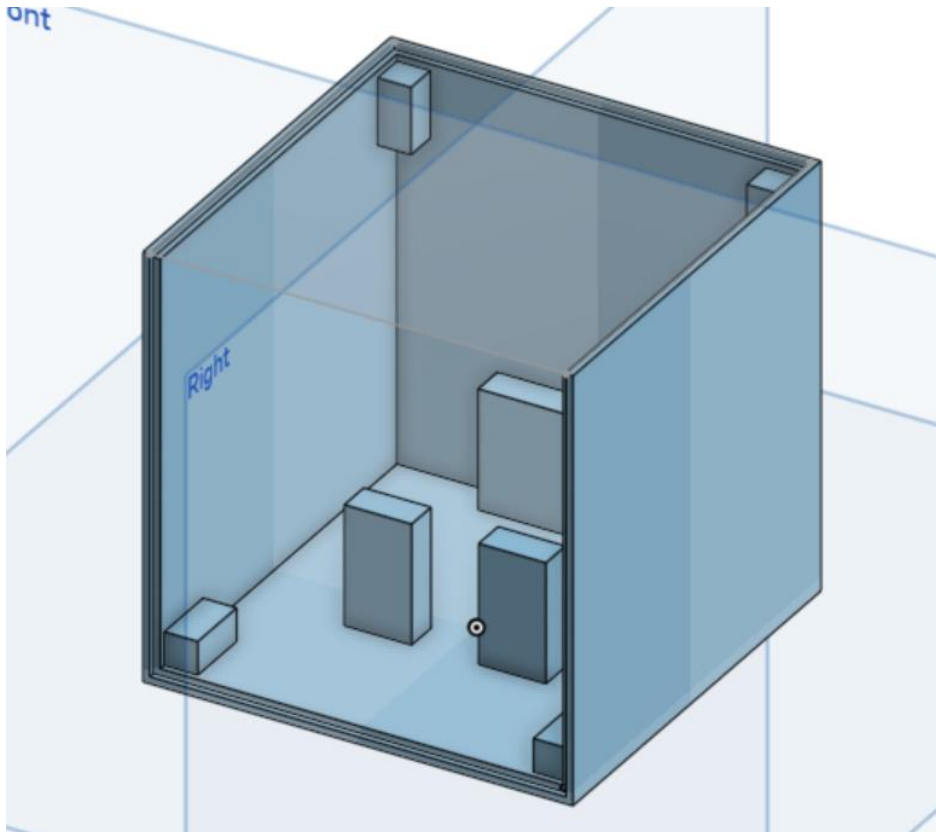
```

Würfel

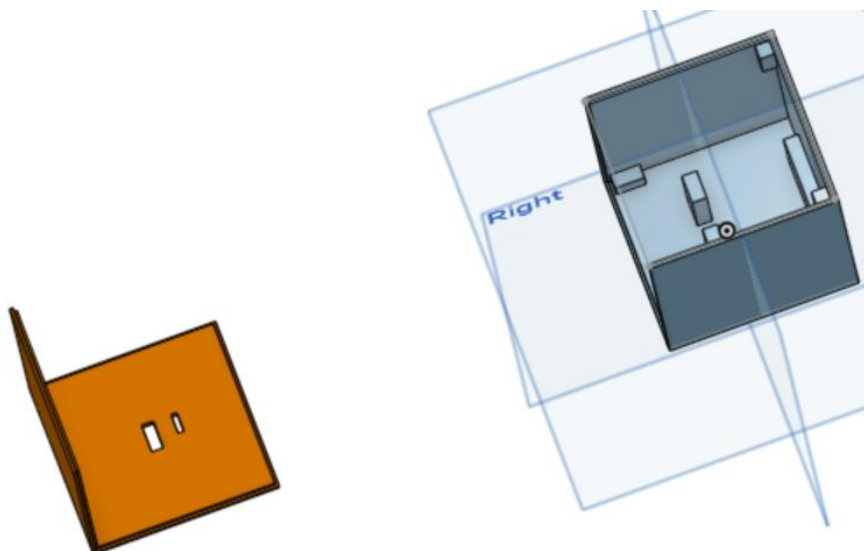
OnShape design

Onshape ist eine CAD-Software zur 3D-Modellierung, die wir benutzt haben, um unseren Würfel zu designen.

Main Würfel:

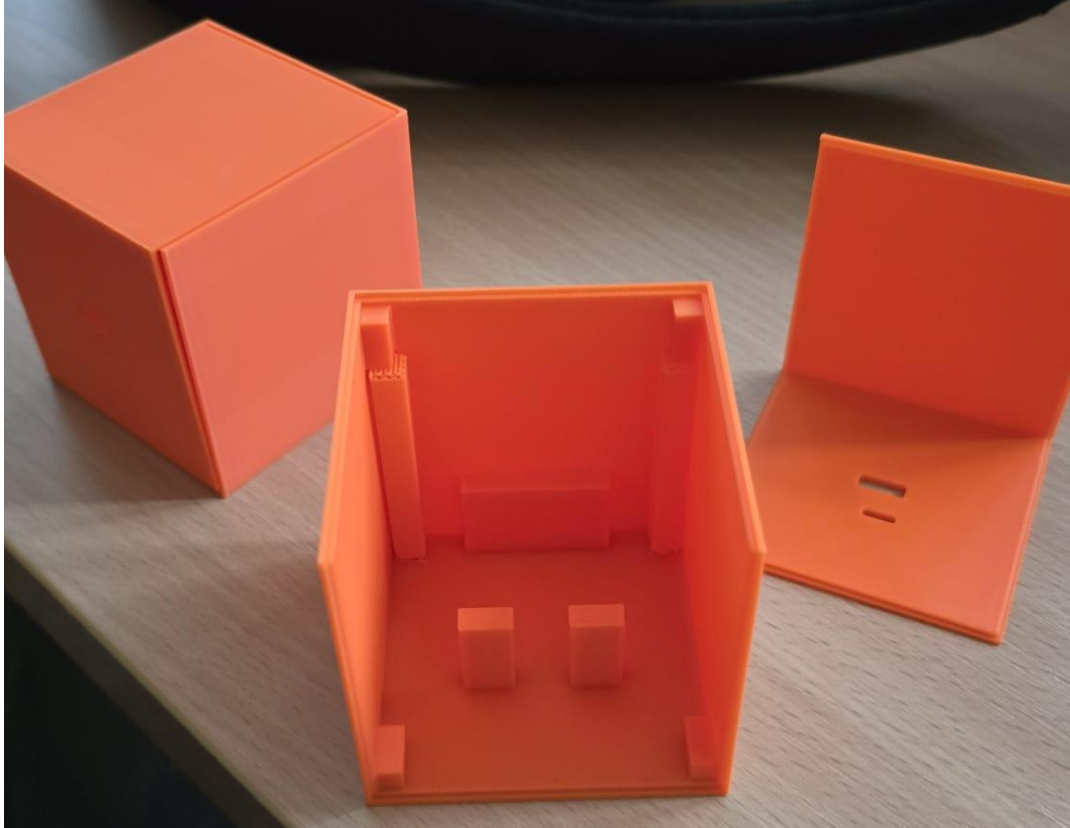


Würfel und Deckel:



3D gedruckt

Wir haben den Würfel doppelt ausgedruckt, damit wir ein Backup haben, falls dem ersten Würfel etwas passieren sollte. Den Deckel kann man mit kleinen Schrauben an den rest des Würfels anschrauben.



Node.js

Installation

Ich habe mir Node.js heruntergeladen:



Run JavaScript Everywhere

Node.js® is a free, open-source, cross-platform JavaScript runtime environment that lets developers create servers, web apps, command line tools and scripts.

Get Node.js®

Get security support
for EOL Node.js versions



Node.js is proudly supported by the partners above and more.

```
Create an HTTP Server Write Tests Read and Hash a File Streams Pipeline Wor

1 // server.mjs
2 import { createServer } from 'node:http';
3
4 const server = createServer((req, res) => {
5   res.writeHead(200, { 'Content-Type': 'text/plain' });
6   res.end('Hello World!\n');
7 });
8
9 // starts a simple http server locally on port 3000
10 server.listen(3000, '127.0.0.1', () => {
11   console.log('Listening on 127.0.0.1:3000');
12 });
13
14 // run with 'node server.mjs'

JavaScript </> Copy to clipboard
```

Learn more what Node.js is able to offer with our Learning materials.

Verbinden

In dem Projektordner, in dem der Rest des Codes für StarFocus ist, habe ich einen neuen Ordner erstellt den ich ,backend‘ genannt habe. Dort drinnen war dann auch dieser Code:

```
const express = require('express');
const cors = require('cors');
```

```
const app = express();
```

```
app.use(express.json());
app.use(cors());
```

```
let currentSide = "Default";
```

```
app.post('/update', (req, res) => {
```

```
  currentSide = req.body.side;
```

```
  console.log("Neue Seite:", currentSide);
```

```
    res.send("OK");
  });

  app.get('/current-side', (req, res) => {
    res.json({ side: currentSide });
  });

  app.listen(3000, () => {
    console.log("Server läuft auf Port 3000");
  });
```

Danach musste ich nur noch in das Terminal im ‚backend‘ Ordner ‚node server.js‘ eingeben und wenn die Nachricht ‚Server läuft auf Port 3000‘ auftaucht läuft der Server.

Wenn alles funktioniert, soll der Würfel die aktuelle Seite in dem Terminal ausgeben:

A screenshot of a terminal window with a black background and white text. The first line is 'Server läuft auf Port 3000'. This is followed by a series of alternating lines: 'Neue Seite: code', 'Neue Seite: reading', 'Neue Seite: code', 'Neue Seite: reading', 'Neue Seite: code', 'Neue Seite: reading', 'Neue Seite: exercise', 'Neue Seite: code', and finally 'Neue Seite: work'.

```
Server läuft auf Port 3000
Neue Seite: code
Neue Seite: reading
Neue Seite: code
Neue Seite: reading
Neue Seite: code
Neue Seite: reading
Neue Seite: exercise
Neue Seite: code
Neue Seite: work
```